Université Paris Cité

Projet I2P2: Calcul de relations spatiales directionnelles à partir de

l'histogramme d'angle

Conception Détaillée

Equipe de projet :

Xie Rolland, Rayan Al Mohaize, Devasenaradjounayagar Damien

Sous la direction de **M. Wendling Laurent**

Année Scolaire : 2022 - 2023

Université Paris Cité

Les informations d'identification du document	*Les éléments de vérification du document*
Référence du document : **L2P2**	***Validé par :*** **Wendling Laurent**
Version du document : **Version 1**	***Validé le :*** **17/03/2023**
Date du document : **17/03/2023**	***Soumis le :*** **17/03/2023**
Auteur(s) : **Xie Rolland** **Rayan Al Mohaize** **Devasenaradjounayagar Damien**	***Type de diffusion :*** **Document électronique (.pdf)**
	Confidentialité : **Réservé aux étudiants UFR Maths-Info de l'université Paris Descartes**

Les éléments d'authentification

Maître d'ouvrage: **Xie Rolland**	***Validé par :*** **Wendling Laurent**
Date / Signature :	***Date / Signature :***

Mots clés : Objets ; Images ; Relations spatiales ; "ARB" ; Barycentre/paire de points ; Histogramme d'angles ; Ensemble flou; Java.

Sommaire

Sommaire 3

1. Introduction 4

2. Guide de lecture 5

2.1. Maîtrise d'œuvre 5

2.2. Maîtrise d'ouvrage 5

3. Concepts de base 6

4. Organisation des tâches 7

5. Planification 8

6. Spécification des besoins 10

7. Cas d'utilisation 11

8. Diagrammes de classes 12

9. Méthode et outils 17

10. Annexes 18

11. Références 19

1. Introduction

Ce document est dédié à la conception détaillée du projet de développement de **calcul de relations spatiales directionnelles à partir de l'histogramme d'angle** en langage JAVA. Celui-ci fait partie des outils qui permettent la lecture d'images. Ce projet vise à déterminer la **relation spatiale entre 2 objets via l'histogramme des angles**.

Pour répondre aux besoins et spécifications fonctionnelles du projet, l'équipe de développement a rédigé un cahier des charges validé par le client et un cahier de recette qui décrit les tests qui seront effectués sur le produit final pour s'assurer de sa conformité aux exigences du cahier des charges.

Ce document de conception détaillée fournit une compréhension claire et complète de notre **représentation de la relation spatiale** et de son fonctionnement interne afin que chacun puisse comprendre son architecture et son potentiel. Il sera utilisé comme référence pour guider l'équipe tout au long du processus de développement et pour assurer la qualité et la cohérence du code.

L'équipe de développement, composée de trois étudiants en deuxième année de licence, collaborera pour concevoir, développer et tester les programmes afin qu'il fonctionne correctement et qu'il apporte le bon résultat. Le document de conception détaillée fournira une vue d'ensemble de l'architecture du système, des classes et des fonctions du programme, ainsi que des tâches qui seront effectuées par l'équipe pour implémenter chaque fonctionnalité.

Chaque classe permet de représenter un élément de la scène ou un élément permettant de générer le rendu (angles, coordonnées, etc.).

2. Guide de lecture

2.1. Maîtrise d'œuvre

La maîtrise d'œuvre rassemble l'ensemble des personnes travaillant à la réalisation du projet. Elle devra impérativement maîtriser les concepts de base du langage de programmation (*Cf. partie 3*). Elle doit aussi décrire les différentes classes et méthodes utilisées par chaque fonctionnalité (*Cf. partie 8*) en détaillant de façon maximale, notamment les arguments de chaque méthode.

Pour ce projet, trois maîtres d'œuvre ont été désignés: Xie Rolland, Rayan Al Mohaize, Devasenaradjounayagar Damien.

Chacun des maîtres d'œuvre se verra être chef de projet durant 3 semaines consécutives avant de céder sa place à un autre. Le premier chef de projet sera Devasenaradjounayagar Damien, puis Rayan Al Mohaize, et enfin Xie Rolland.

Les maîtres d'œuvre sont encadrés et guidés par la maîtrise d'ouvrage. Elle est représentée par le responsable de projet : M. Wendling Laurent.

2.2. Maîtrise d'ouvrage

Le maître d'ouvrage est le client ayant commandité le projet. Les clients doivent pouvoir comprendre au mieux le code pour chaque fonctionnalité du programme. Pour cela, la lecture de ce document doit leur permettre de comprendre l'utilité et le rôle de chacune des méthodes utilisées en fonction des classes présentes pour l'élaboration du projet. (*Cf. partie 9).*

La personne bénéficiant des résultats du projet est M. Wendling, qui est donc le maître d'ouvrage. M. Wendling sera également notre encadrant tout au long de la réalisation du projet. Le rôle de l'encadrant est de guider le groupe et de vérifier le bon déroulement du projet, tout en donnant en parallèle des conseils et directives. Une réunion hebdomadaire est prévue pour cela chaque vendredi à 19h00.

3. Concepts de base

Les concepts clés liés aux **Relations spatiales**, en rapport avec la conception détaillée, incluent les:

CLASSE [1] : Une classe permet de regrouper dans une même entité des données et fonctions membres (appelées aussi méthodes) permettant de manipuler ces données. Une classe possède plusieurs facettes :
- un type qui décrit une structure de données (ses variables d'instance) et un comportement (ses méthodes).
- un module pour décomposer un programme en sous-programmes.
- un générateur d'objets (grâce à ses constructeurs).

MÉTHODE [2]: Une méthode est une fonction faisant partie d'une classe. Les méthodes déterminent le comportement des instances de la classe quand elles reçoivent un message.

STRUCTURE DE DONNÉES [3] : En informatique, une structure de données est une manière d'organiser les données pour les traiter plus facilement. Une structure de données est une mise en œuvre concrète d'un type abstrait.

ENSEMBLE FLOU[4] : Un ensemble flou est un concept mathématique utilisé pour représenter la gradation ou l'incertitude dans la définition de l'appartenance d'un élément à un ensemble. Contrairement à un ensemble classique, qui peut soit appartenir ou ne pas appartenir à un ensemble donné, un élément peut appartenir à un ensemble flou avec un degré de pertinence compris entre 0 et 1.
Via une analyse d'angles entre deux ensembles flous on va pouvoir déterminer la position relative d'un ensemble par rapport à un autre.

4. Organisation des tâches

Dans ce projet, les différentes fonctionnalités à réaliser sont les suivantes :

-**Lecture d'image**: La première étape sera de faire une lecture d'image (une image qui sera fournie). Cette étape consiste à lire une image et extraire les informations nécessaires à la création de la scène, c'est-à-dire qu'il pourra afficher un objet seul tout en gardant la position de l'objet ainsi que sa couleur. Il s'agit d'une des étapes les plus importantes pour la construction de la scène car c'est celui qui posera les termes de base afin que la suite des étapes fonctionne correctement. Elle permet de définir chaque objet de l'image de manière assez précise.

-**Pixel**: L'étape de pixel consiste à déterminer pour chaque pixel dans une image, sa couleur et sa position. Cette étape est essentielle puisque chaque image est constituée de milliers de pixels, et donc la détermination de chaque pixel permettra de distinguer de façon précise un objet et sa délimitation par rapport à un autre objet. On pourra par la suite les séparer et les stocker dans un tableau.

-**Angle**: L'angle est essentiel pour la réalisation de notre projet. Il consistera aux calculs d'angle d'un objet par rapport à un autre objet. Afin d'apporter plus de précision dans ces calculs, il fera usage du pixel pour déterminer l'angle d'un pixel appartenant un objet A, par rapport à tous les autres pixels de l'objet B. Il permet notamment le calcul du barycentre pour chaque objet via leur centre de gravité.

-**Histogramme**: L'étape de l'histogramme est en quelque sorte la "finalisation" du programme. En effet, avec les résultats du calculs d'angles, on pourra afficher l'histogramme de tous les angles et donc pouvoir établir une échelle permettant de définir les degrés des positions en fonction de l'histogramme.

-**Fichier**: Cette étape consiste à enregistrer dans un fichier, qui sera créé automatiquement, le résultat de l'affichage de l'histogramme ou de l'angle du barycentre afin d'avoir une trace dans son dossier et que cela soit plus simple à visualiser. Cette étape peut être utile dans le cas où le client rencontre une erreur et doit réinitialiser le programme.

-**Affichage**: La position relative de l'objet choisi sera affiché à l'écran ainsi que l'interprétation des résultats, dépendant des positions, sous forme de quatres phrases. Une phrase par direction (à droite, à gauche, en haut, en bas).

5. Planification

Dans le cadre de la phase de conception détaillée d'un projet de relation spatiale directionnelle en JAVA, il est important de mettre en place une planification efficace pour assurer une progression régulière et significative du projet. Pour cela, il est recommandé de suivre une méthode agile qui permet une adaptation constante aux besoins et contraintes du projet. Chaque semaine, l'équipe se réunira pour définir des objectifs hebdomadaires clairs, précis et réalisables en un temps défini, et prendre en compte de ce que l'encadrant attend de nous pour chaque semaine avant une autre réunion.

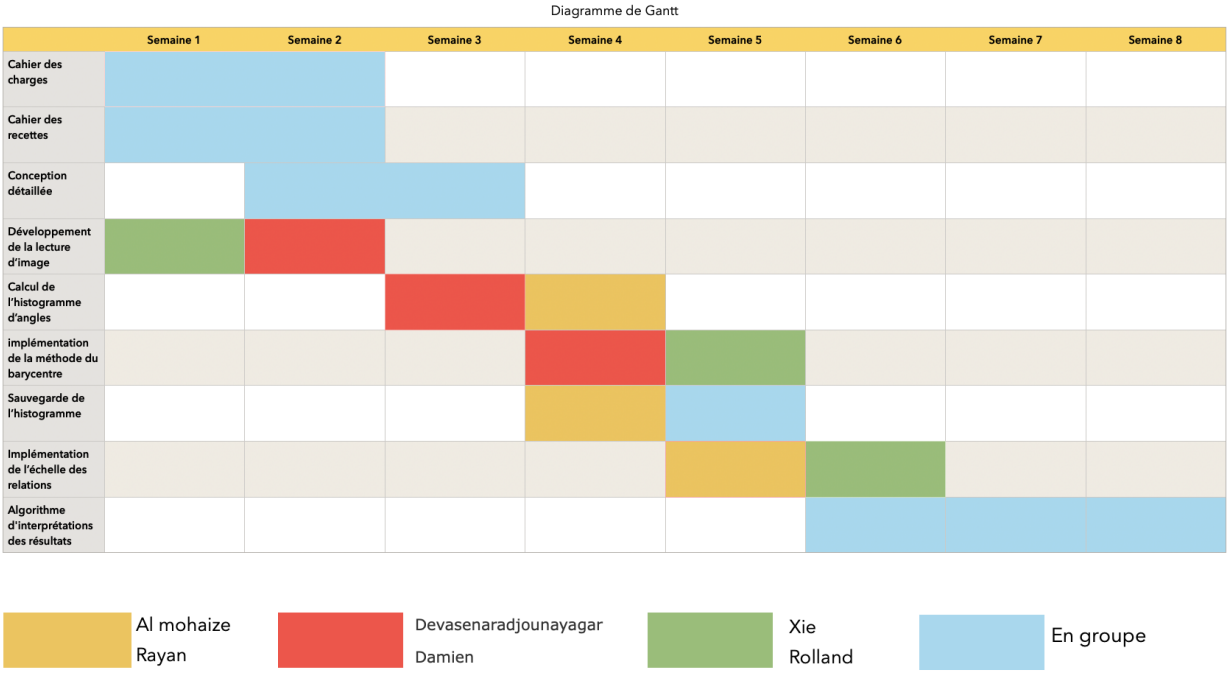
En fonction des objectifs fixés, l'équipe doit attribuer des tâches spécifiques à chaque membre en fonction de ses compétences et de ses disponibilités. Ils peuvent également travailler à plusieurs sur une même tâche qui peut être complexe afin d'accroître l'efficacité de l'avancement du projet. Il est donc crucial de travailler en étroite collaboration tout au long du projet, en communiquant régulièrement pour s'assurer que chacun est sur la même longueur d'onde et pour résoudre rapidement les problèmes qui se posent.

Si des problèmes surviennent, l'équipe doit travailler ensemble pour les résoudre rapidement et efficacement, sans les laisser s'accumuler, car cela pourrait ralentir considérablement la progression du projet.

Chaque semaine, l'équipe doit faire une vérification de l'état d'avancement du projet pour s'assurer que les objectifs ont été atteints et que le projet est sur la bonne voie, afin d'éviter des hors sujets. Si des ajustements doivent être apportés, ils peuvent être discutés et mis en place pour la semaine suivante.

En suivant cette méthode, l'équipe pourra travailler efficacement, résoudre les problèmes rapidement et progresser régulièrement vers la réalisation du projet.

Gestion des tâches



<u>Figure 1 :</u> Diagramme de Gantt

6. Spécification des besoins

La **relation spatiale directionnelle** présente des besoins pour satisfaire le client et garantir sa satisfaction.

Le programme doit pouvoir :

-Sauvegarde de chaque région dans un objet.
-Établir un calcul exhaustif de tous les angles d'un point par rapport à ceux d'un autre objet.
-Fournir un rendu de l'histogramme des angles.
-Sauvegarder l'histogramme.

Les critères permettant de considérer l'application comme acceptable sont :

-Forme du traitement: L'application doit pouvoir traiter des images JPG ou format similaire qui sera fourni par le client.
-Précision: L'application doit être précise et cohérente. Les erreurs doivent être minimisées autant que possible.
-Rapidité: L'application doit être rapide et efficace, de sorte qu'elle puisse être utilisée en temps réel.
-Interprétabilité: L'application doit être facile à interpréter et à comprendre. Les résultats doivent être présentés de manière claire et compréhensible.

L'utilisateur doit pouvoir agir sur des éléments et proposer des scènes différentes :

- Les images à traiter
- Décision du choix entre l'objet A et l'objet B pour l'affichage
- La méthode à utiliser

7. Cas d’utilisation

Le traitement du cas d’utilisation pour ce projet est plutôt simple. On se placera du point de vue de l’utilisateur qui doit lui soumettre une photo/image :

L’utilisateur devra proposer une image à un format valide(.PNG ou similaire), ensuite il devra choisir un objet, via sa couleur, pour lequel il voudra déterminer la position par rapport au second objet qui sera automatiquement défini comme référent. Enfin après le choix de la méthode, une brève description sera affichée à l’écran donnant la relation et le degré de relation entre l’objet choisi et le référent, accompagné de l’histogramme d’angles dans un fichier pour soutenir le résultat.

8. Diagrammes de classes

Un diagramme de classes fournit une vue globale d'un système en présentant ses classes, interfaces et collaborations, et les relations entre elles dans le langage de modélisation unifié (UML). Les diagrammes de classes sont statiques : ils affichent ce qui interagit mais pas ce qu'il se passe pendant l'interaction.

Les classes permettant créer une **relations spatiales directionnelles à partir de l'histogramme d'angles** seront au nombre de 3 :

● Classe Couleur

- Description : La classe Couleur sert à définir les constantes de couleurs de nos images.

- Attributs :

1. public static final int VIOLET : Représente la couleur constante violette.
2. public static final int JAUNE : Représente la couleur constante jaune.
3. public static final int VERT : Représente la couleur constante verte.

- Méthodes :

Aucune.

● Classe Pixel

- Description : La classe Pixel à la responsabilité de retourner les couleurs ainsi que les coordonnées de chaque pixel, et permet le calcul des angles et celle du barycentre. Il affichera ainsi tous les angles calculés. Cette classe est utilisée pour les calculs d'angles entre un pixel d'un objet avec tous les autres pixels de l'autre objet.

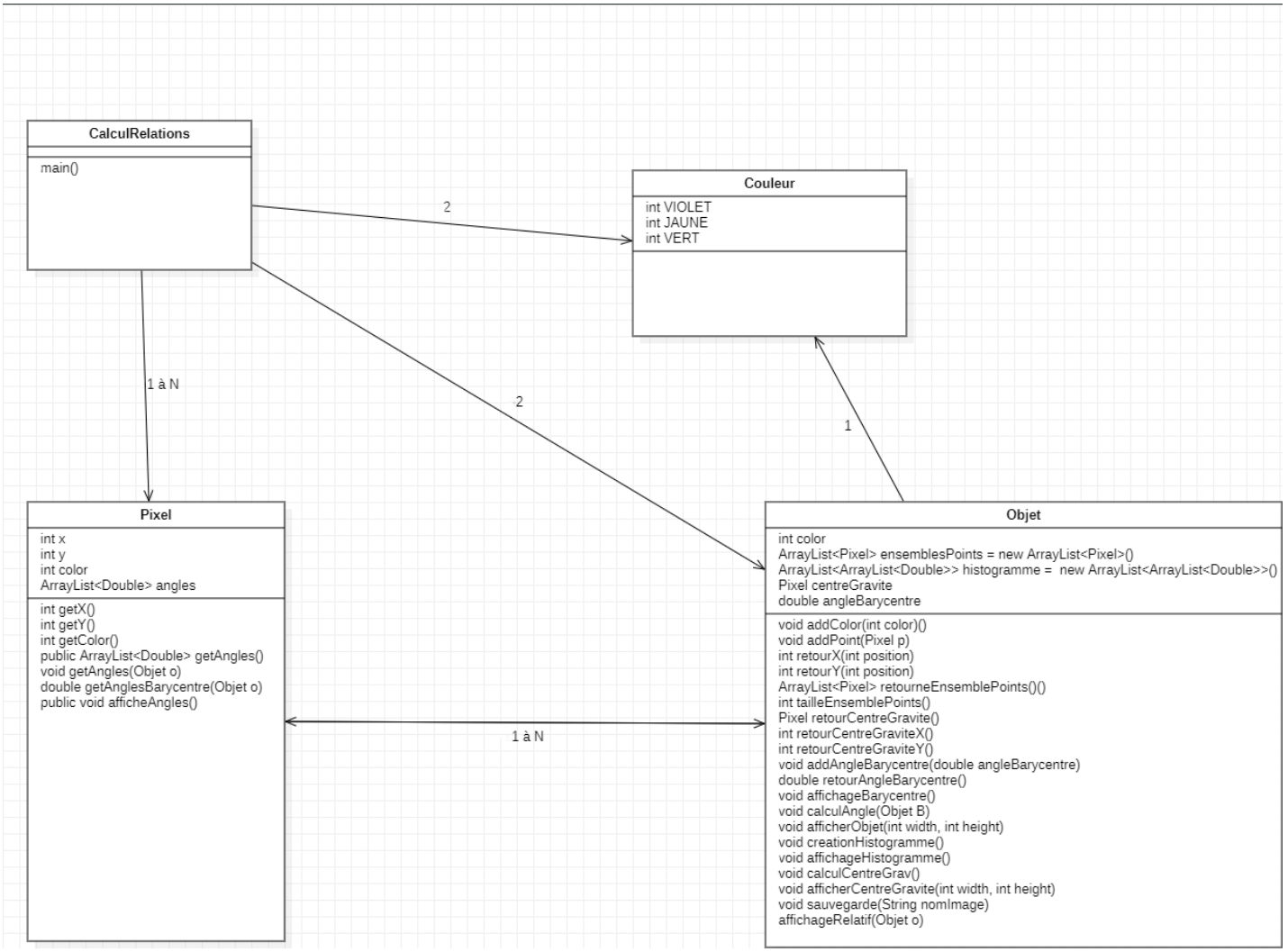
- Attributs :
    1. private int x,y : Les variables des points de la classe.
    2. private int color : Les variables qui définissent une couleur de la classe.
    3. private ArrayList<Double> angles : ArrayList permettant de stocker les valeurs des angles.

- Méthodes :
    1. public int getX() : Retourne la valeur de X.
    2. public int getY() : Retourne la valeur de Y.
    3. public int getColor() : Retourne la couleur souhaitée.
    4. public ArrayList<Double> getAngles() : Retourne l'ensemble des angles qui sont stockés dedans.
    5. public void getAngles(Objet o) : Méthode permettant de faire le calcul d'angle entre 1 point et tout le reste des points.
    6. public double getAnglesBarycentre(Objet o) : Méthode permettant de calculer l'angle du barycentre avec un objet passé en argument.
    7. public void afficheAngles() : Méthode permettant l'affichage des angles calculés grâce à la méthode "getAngles(Objet o)".

● Classe Objet

- Description : La classe Objet a la responsabilité de stocker ses points dans le but de pouvoir créer un histogramme et de l'afficher. Il s'occupe également du calcul et de l'affichage du centre de gravité. Il sauvegarde tout cela dans un fichier. Cette classe est une classe assez polyvalente, elle est utilisée pour récolter les informations d'un objet et réaliser un histogramme de celui-ci.
- Attributs :
    1. private int color : Variable de la couleur des points de l'objet.
    2. private ArrayList<Pixel> ensemblesPoints = new ArrayList<Pixel>() : Liste de tous les points de l'objet.
    3. private ArrayList<ArrayList<Double>> histogramme = new ArrayList<ArrayList<Double>>() : ArrayList d'ArrayList qui va stocker pour chaque angle le nombre de points que l'objet a.
    4. private Pixel centreGravite : Variable du pixel de centre de gravité de l'objet.
    5. private double angleBarycentre : Variable de l'angle du Barycentre.
- Méthodes :
    1. public void addColor(int color): Permet l'ajout d'une couleur passé en argument.
    2. public void addPoint(Pixel p): Permet l'ajout d'un point passé en argument.
    3. public int retourX(int position): Retourne la variable X d'un point à l'aide de sa position passé en argument.
    4. public int retourY(int position): Retourne la variable Y d'un point à l'aide de sa position passé en argument.
    5. public ArrayList<Pixel> retourneEnsemblePoints(): Retourne l'ensemble des points présent dans le ArrayList.
    6. public int tailleEnsemblePoints(): Retourne la taille de ArrayList.
    7. public Pixel retourCentreGravite(): Retourne le centre de gravité de type Pixel.
    8. public int retourCentreGraviteX(): Retourne le point X du centre de gravité.
    9. public int retourCentreGraviteY(): Retourne le point Y du centre de gravité.
    10. public void addAngleBarycentre(double angleBarycentre): Permet l'ajout de l'angle passé en argument en remplacant la variable "angleBarycentre" par celle passé en argument.
    11. public double retourAngleBarycentre(): Retourne la variable "angleBarycentre".
    12. public void affichageBarycentre(): Permet l'affichage de l'angle du barycentre.
    13. public void calculAngle(Objet B): Permet d'obtenir l'angle entre un point passé en argument avec les autres angles.

14. public void afficherObjet(int width,int height): Permet l’affichage seul d’un objet à l’aide de ces points ainsi que les variables qui sont en argument.
15. public void creationHistogramme(): Permet la création de l’histogramme via les points présent dans ArrayList.
16. public void affichageHistogramme(): Affichage de l’histogramme sur la console.
17. public void calculCentreGrav(): Permet le calcul du centre de gravité d’un Objet.
18. public void afficherCentreGravite(int width,int height): Affichage du centre de gravité. Il sera affiché directement dans une fenêtre.
19. public void sauvegarde(String nomImage): Permet la sauvegarde de l’histogramme et de l’angle du barycentre, dans un fichier qui sera créé automatiquement, afin de mieux visualiser et d’avoir des traces du résultat.
20. public void affichageRelatif(Objet o): Affiche la position relative d’un objet par rapport à l'autre selon les 4 directions différentes.



<u>Figure 2 :</u> Diagramme de Classe

9. Méthode et outils

Le langage de programmation choisi pour ce projet est le langage JAVA, un langage polyvalent et performant et couramment utilisé pour le développement d'applications en temps réel, y compris les applications 3D.

Pour faciliter l'écriture, la compilation et le débogage de notre code, nous avons choisi Eclipse et IntelliJ IDEA, des environnement de développement intégré et multi plateforme, qui offrent des fonctionnalités utiles pour le développement de logiciels tels que la coloration syntaxique, la gestion de projets et la détection d'erreurs de syntaxe.

Afin de gérer efficacement les modifications de notre code source et faciliter la collaboration entre les membres de l'équipe de développement, nous avons choisi SVN (Subversion) comme système de contrôle de version. SVN est un système de contrôle de version populaire qui offre une gestion efficace des versions et des branches, ainsi qu'une collaboration simplifiée entre les membres de l'équipe.

10. Annexes

<u>Figure 1 :</u> Diagramme de Gantt

<u>Figure 2 :</u> Diagramme de Class

11. Références

1. Classe: https://blog.paumard.org/cours/java/chap04-structure-classe-classe.html
2. Méthode: https://web.maths.unsw.edu.au/~lafaye/CCM/java/javafonc.htm
3. Fichier: https://www.baeldung.com/java-filename-without-extension
4. Histogramme d'angle: https://www.researchgate.net/figure/A-partir-de-deux-objets-A-et-B-de-limage-a-un-histogramme-dangles-peutetre_fig12_216487536
5. System setOut(): https://www.educative.io/answers/what-is-the-system-setout-function-in-java
6. couleur rgb : https://teaching.csse.uwa.edu.au/units/CITS1001/colorinfo.html
7. ArrayList: https://www.w3schools.com/java/java_arraylist.asp
8. Calcul d'angle: https://www.w3schools.blog/angle-between-two-points#:~:text=double%20angle%20%3D%20atan2(y2%20-,*%20180%20%2F%20PI%3B".&text=between%20two%20points-,double%20angle%20%3D%20atan2(y2%20-%20y1%2C%20x2%20-%20x1,*%20180%20%2F%20PI%3B"
9. GeoGebra: https://www.geogebra.org/classic?lang=en
10. DateTimeFormatter: https://www.baeldung.com/java-datetimeformatter